CST8509 Part II

Today's Agenda

- Midterm results
- Gazebo
- RViz
- Dynamic Programming
- Monte Carlo

Toolbox

- Today we add Gazebo and Rviz to our RL toolbox:
  - Gymnasium (Environments)
  - Stable-Baselines3 (Algorithm implementations/ agents)
  - Gazebo (?? Simulation)
  - Rviz (?? Robot Visualization)

Gazebo

Gazebo is a simulator: https://gazebosim.org/home

- Simulates environments
- Simulates robots
- Plugin-based physics, rendering, and GUI libraries
- ROS integration

Gazebo Versions

- Gazebo versions can be confusing, because there are two choices:
    - Classic Gazebo (version 11), or Gazebo11 or Gazebo-11 or Classic Gazebo
    - Ignition Gazebo (version Fortress, Harmonic, etc). Note that Ignition Gazebo has been renamed to just Gazebo! This Gazebo choice can be called Gazebo, Ignition Gazebo, Gazebo Sim.
- We will use Classic Gazebo 11 for Lab 4
- To Install Classic Gazebo 11 on Ubuntu-desktop 22.04

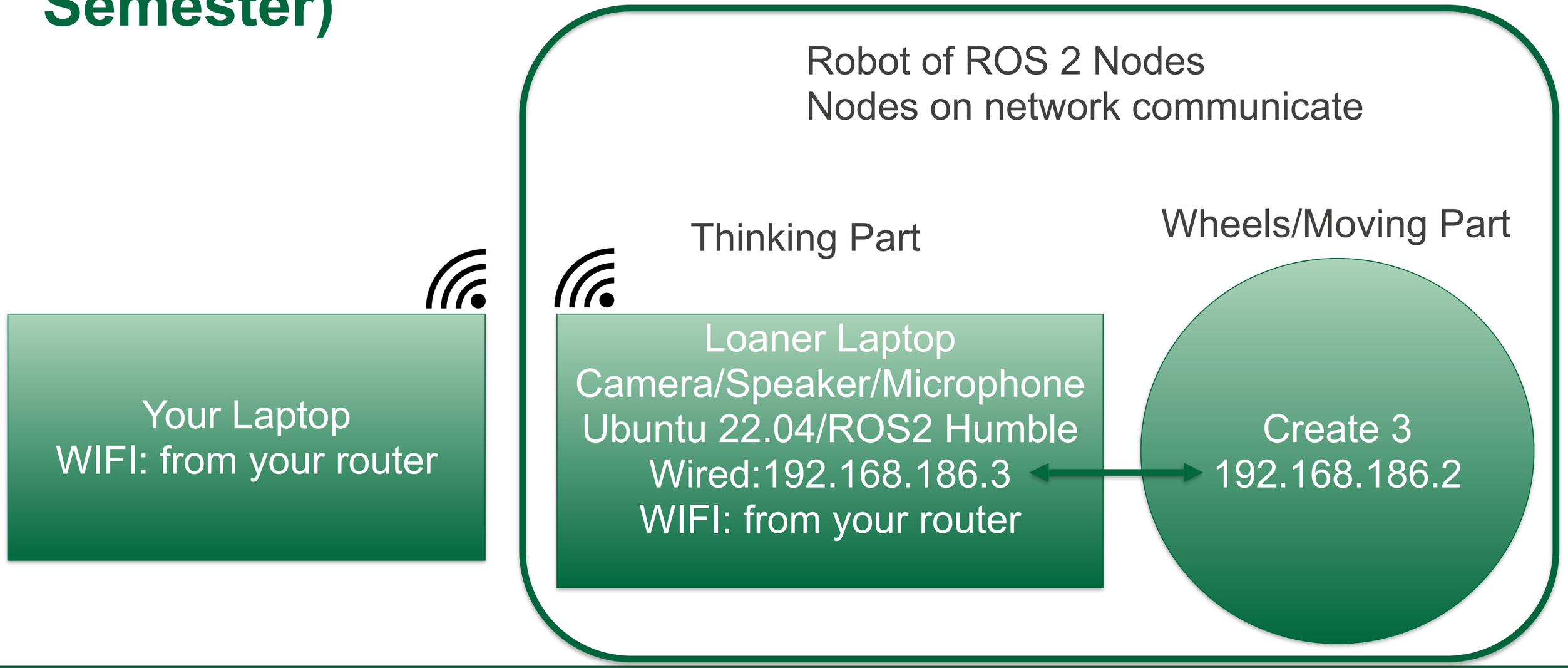
```
sudo apt update && sudo apt upgrade
curl -sSL http://get.gazebosim.org | sh
```

RL with Create3 robot

First let's imagine applying RL to our Create3 robots from CST8504

- Environment:
  - Image publisher
  - Recording publisher (we'll ignore voice commands for now)
- Agent:
  - Hands Module
  - Move Module
  - Actions: Twist Messages from Move Module
- Reward:
  - Need to add a reward generator
  - Need to be careful with step duration, moving target, etc

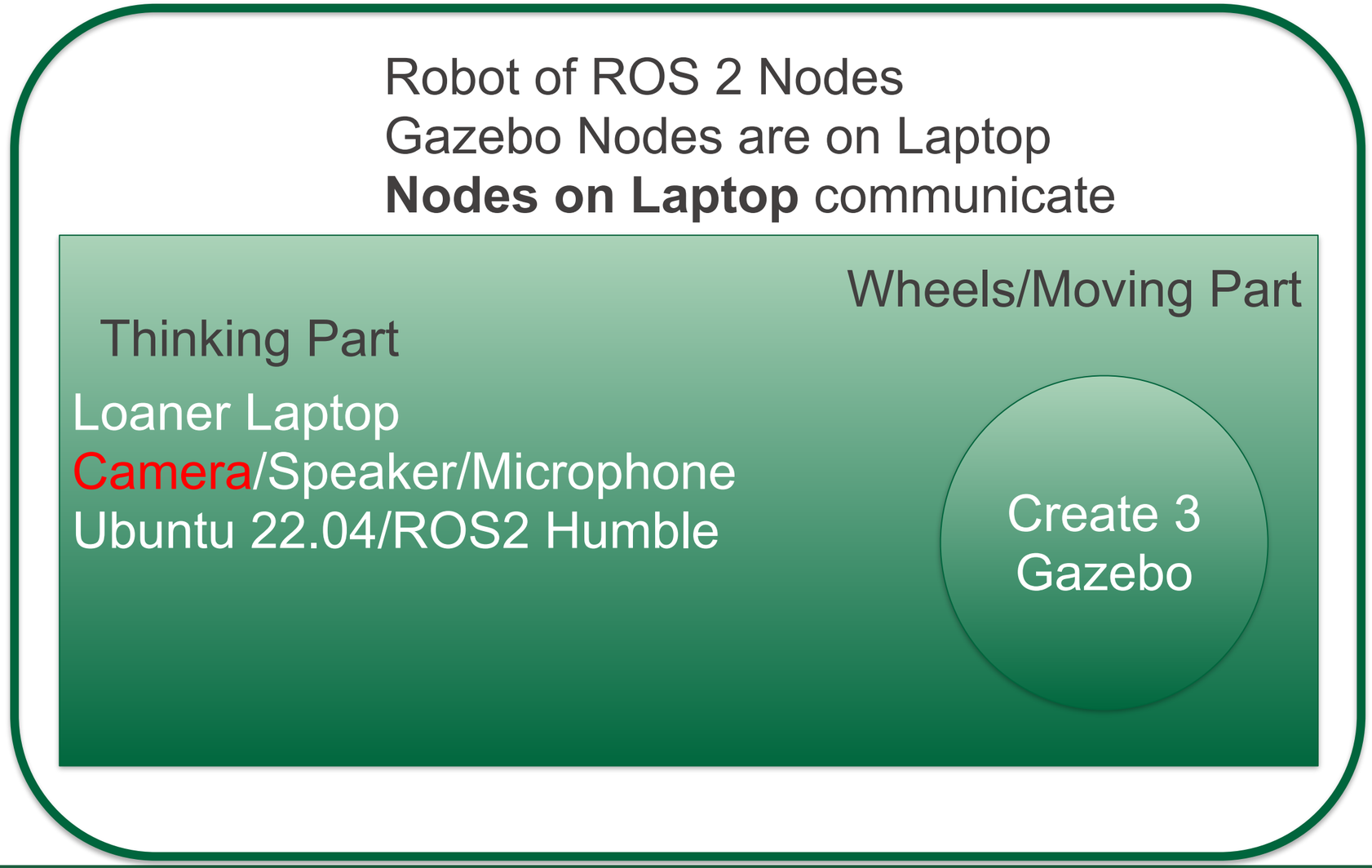
Our Create 3 Setup (Last Semester)



Our Create 3 Setup (Simulated)

This still uses the Laptop Camera (remember we made the turtlesim move with our actual hand in front of the laptop camera)

Lab4 involves adding a Virtual Gazebo Camera that sees the Gazebo world



Why is Gazebo so important for our RL?

Assuming we work out the details RL with the Create3, how will we train the agent?

- Q: Who is going to move their hand around during training?
    - What if training takes 3 days, won't that person get tired?
- Q: How do we maintain robot safety in early training stages?
    - During training, the robot tries a dangerous action?
- A: If the hand person and the robot are simulated, they can't get tired or hurt
    - Gazebo can simulate the robot, the person/hand, and the environment

RL Diagrams so far

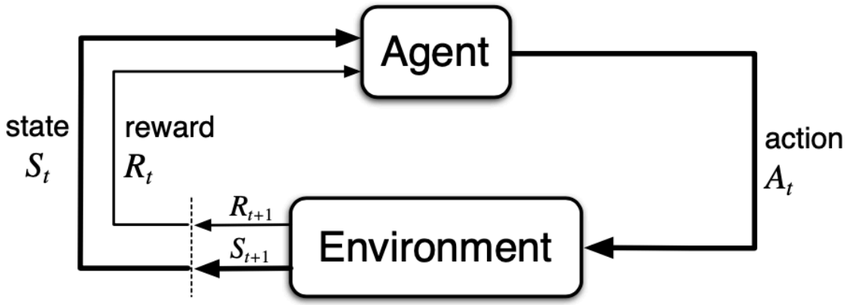
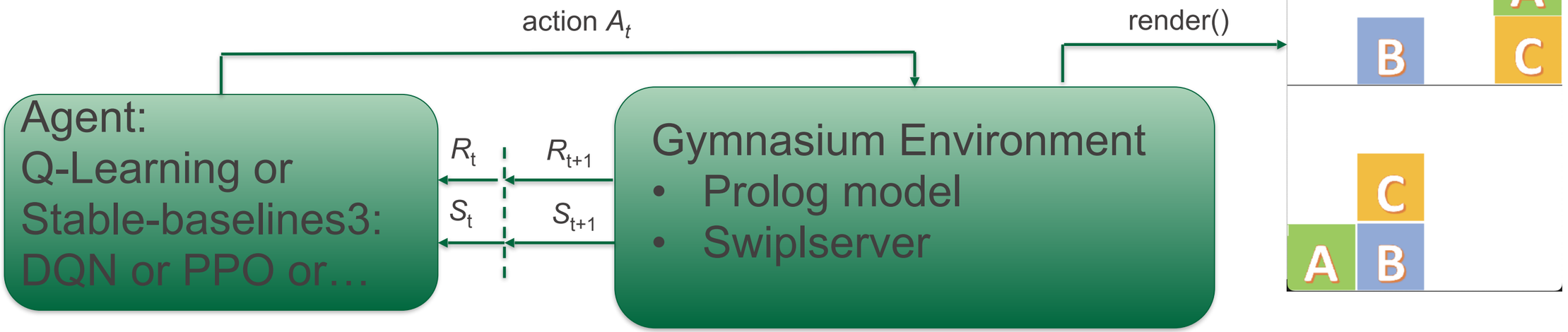


Figure 3.1: The agent–environment interaction in a Markov decision process.

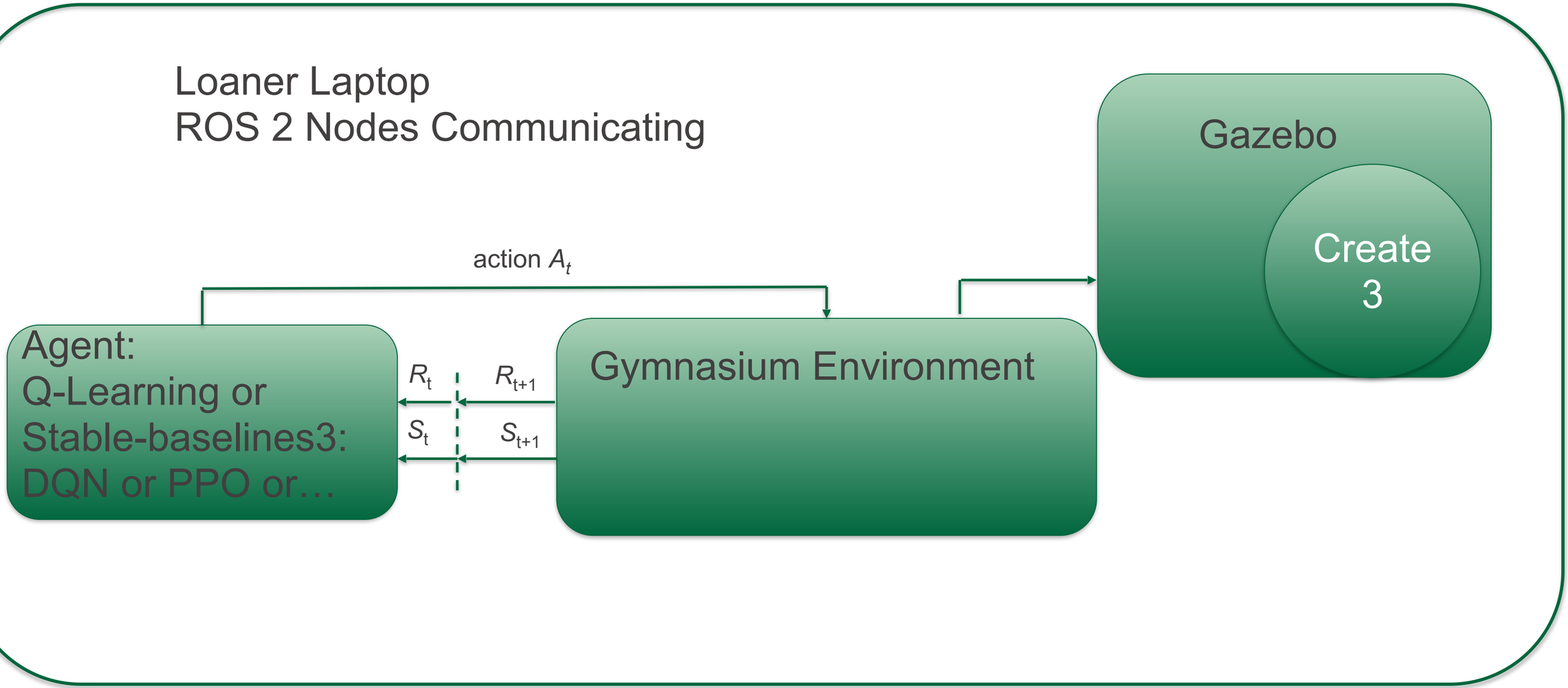


Where do ROS 2 and Gazebo fit into this picture?

- The Robot Operating System (ROS 1 and ROS 2) is a set of software libraries and tools for building robot applications.

- Gazebo supports ROS integration
    - Gazebo version of Create3 publishes on ROS 2 topics

RL Diagrams with virtual Create 3



iRobot Create 3 Simulator

iRobot has already done the work of creating the Gazebo simulated **Create 3**

AWS has already done the work of creating the Gazebo **AWS small house world**

iRobot Create 3 Simulator:

- https://iroboteducation.github.io/create3_docs/sim/setup/
- Instructions for installing and running (we use Classic Gazebo 11):
    - https://github.com/iRobotEducation/create3_sim

Adding a Camera to Create3

Unified Robotic Description Format (**URDF**)

- File format used in ROS
- File format used for Create 3 simulator
- Build process will convert this to SDF

Simulation Description Format (**SDF**)

- New format created for use in Gazebo
- Intended to address shortcomings of URDF

Xacro

- XML macro language (both URDF and SDF are XML)

URDF Details

We won't be digging deeper than just adding our camera in Lab 4, but for those who want to dig deeper:

https://docs.ros.org/en/iron/Tutorials/Intermediate/URDF/Building-a-Visual-Robot-Model-with-URDF-from-Scratch.html

Rviz

Graphical interface for viewing robot, sensor data, maps, and more.

https://turtlebot.github.io/turtlebot4-user-manual/software/rviz.html

Dynamic Programming

- DP is a collection of algorithms to compute optimal policies **given a perfect model**
- Policy Evaluation: compute the state-value function v for an arbitrary policy
  - **Iteratively** follow the policy, use Bellman Equation to update value function (**keep doing this** until differences from previous are "small")
- Policy Improvement: make it greedy with respect to the new value function
- Policy Iteration: iterate policy evaluation and improvement
  - drawback is that each of its iterations involves policy evaluation
- Value Iteration: like Policy Iteration but…
  - Difference is that Policy Evaluation is stopped after one iteration

Dynamic Programming

Jupyter Notebook implements iterative value iteration:

`dynamic.ipynb`

Monte Carlo

Dynamic Programming required knowing the model in advance (results of actions)

Monte Carlo methods are based on averaging sample returns

- Complete an episode and afterwards, record the returns such that returns are averaged over time

All episodes must terminate

State looping: distinguish between visits to the same state in a single episode:

- first-visit methods
- every-visit methods

MC methods

How to ensure every action is tried:

- Exploring starts: try a random state/action pair at the beginning of each episode
    - Can't be used with an environment
- Consider only policies that are stochastic with a nonzero probability of selecting all actions in each state
- epsilon-soft policies: every action has a chance
- epsilon-greedy: example of epsilon-soft policy

Monte Carlo

The agent goes to the end of an episode without changing the Q-table

After an episode ends, the agent backs up and updates the q-table (just until the first change)

monte_carlo.ipynb implements algorithm on Page 111 of Sutton + Barto textbook